



# CONCORRÊNCIA E MULTITHREADING

AULA 09 - MÃO NA MASSA



<https://github.com/eldermoraes/unipds/tree/main/Concorrencia-Multithreading>

## AULA 02 - EXECUTORS E GERENCIAMENTO MODERNO DE THREADS

- ❑ Exemplo 2.1: Cargas de Trabalho CPU-Bound com um Pool Fixo de Threads de Plataforma
  - ❑ Para tarefas que consomem intensivamente a CPU, como cálculos matemáticos complexos ou processamento de imagem, o número de threads deve ser cuidadosamente alinhado com o hardware disponível para evitar perdas de performance.
- ❑ Notas
  - ❑ Este código demonstra o caso de uso clássico para um ExecutorService com um número fixo de threads de plataforma (newFixedThreadPool). A estratégia aqui é limitar o número de threads ao número de núcleos de CPU disponíveis. A razão para isso é que tarefas CPU-bound mantêm o processador constantemente ocupado. Se criarmos mais threads do que núcleos, o sistema operacional será forçado a realizar trocas de contexto frequentes, pausando uma thread para executar outra. Essa troca de contexto tem um custo (overhead) e, em vez de acelerar, degrada a performance geral do sistema. Este exemplo estabelece o modelo de "gerenciamento de recursos escassos", onde as threads de plataforma são vistas como um recurso valioso que precisa ser cuidadosamente alocado.

## AULA 02 - EXECUTORS E GERENCIAMENTO MODERNO DE THREADS

### ❑ Exemplo 2.2: Alta Vazão de I/O com `newVirtualThreadPoolExecutor`

- ❑ Em contraste com as tarefas CPU-bound, as tarefas I/O-bound passam a maior parte do tempo esperando por respostas da rede, do disco ou de outros sistemas. As threads virtuais foram projetadas especificamente para escalar aplicações com um grande número dessas tarefas.

### ❑ Notas

- ❑ Este exemplo demonstra a revolução introduzida pelo Projeto Loom. O `Executors.newVirtualThreadPoolExecutor()` cria uma nova thread virtual para cada uma das 100.000 tarefas submetidas. Tentar fazer isso com threads de plataforma esgotaria a memória do sistema rapidamente.
- ❑ O mecanismo chave é que as threads virtuais são gerenciadas pela JVM, não diretamente pelo sistema operacional. Elas são "montadas" em um pequeno pool de threads de plataforma chamadas de carrier threads. Quando uma thread virtual executa uma operação de I/O bloqueante (como o `Thread.sleep()` aqui), a JVM a "desmonta" da carrier thread, liberando a thread de plataforma para executar outra thread virtual. Assim que a operação de I/O é concluída, a JVM "remonta" a thread virtual em uma carrier thread disponível para continuar sua execução. Esse processo permite que um pequeno número de threads de plataforma gerencie milhões de threads virtuais concorrentes, alcançando uma vazão (throughput) massiva para cargas de trabalho I/O-bound.
- ❑ A existência do `newVirtualThreadPoolExecutor` representa uma mudança filosófica no propósito da classe `Executors`. Anteriormente, seu papel era criar pools para limitar a concorrência e reutilizar um recurso escasso. Agora, sua funcionalidade mais poderosa é criar um executor que permite concorrência massiva, tratando threads como um conceito lógico abundante. Isso reflete uma transição de um modelo de concorrência restrito por recursos para um modelo centrado na tarefa.

## AULA 02 - EXECUTORS E GERENCIAMENTO MODERNO DE THREADS

- ❑ Exemplo 2.3: Agendamento de Tarefas com ScheduledThreadPoolExecutor
  - ❑ Para tarefas que precisam ser executadas em um momento futuro ou repetidamente em intervalos regulares, o ScheduledThreadPoolExecutor é a ferramenta ideal.
- ❑ Notas
  - ❑ Este código ilustra o ScheduledThreadPoolExecutor, que permite agendar a execução de tarefas. É importante distinguir scheduleAtFixedRate de scheduleWithFixedDelay:
    - ❑ scheduleAtFixedRate(task, initialDelay, period, unit): Tenta executar a tarefa em intervalos fixos. Se uma execução levar mais tempo que o período, a próxima pode começar imediatamente após a anterior terminar, levando a um "agrupamento" de execuções para tentar "alcançar" o cronograma.
    - ❑ scheduleWithFixedDelay(task, initialDelay, delay, unit): Garante um atraso fixo entre o término de uma execução e o início da próxima. É mais seguro se a duração da tarefa for variável ou se for crucial evitar execuções sobrepostas.
- ❑ Casos de uso comuns incluem verificações de saúde de sistemas, limpeza periódica de dados ou geração de relatórios agendados. O ScheduledThreadPoolExecutor é preferível ao antigo java.util.Timer por ser mais robusto e flexível, utilizando um pool de threads para evitar que uma tarefa de longa duração bloqueie as outras.

## AULA 02 - EXECUTORS E GERENCIAMENTO MODERNO DE THREADS

### ❑ Exemplo 2.4: Boas Práticas para o Ciclo de Vida do ExecutorService

- ❑ Um erro comum em aplicações concorrentes é não desligar adequadamente os ExecutorService. Isso pode causar vazamentos de recursos e impedir que a JVM finalize.

### ❑ Notas

- ❑ Este exemplo demonstra a maneira correta de gerenciar o ciclo de vida de um ExecutorService, conforme recomendado na aula. As threads criadas por um ExecutorService (a menos que uma ThreadFactory customizada seja usada) não são threads daemon. Isso significa que, se o executor não for explicitamente desligado, a JVM não terminará, mesmo que a thread main tenha concluído seu trabalho.
- ❑ shutdown(): Inicia um desligamento ordenado. O executor não aceita novas tarefas, mas as tarefas já submetidas serão concluídas.
- ❑ awaitTermination(...): Bloqueia a thread atual até que todas as tarefas tenham sido concluídas após uma chamada de shutdown, ou até que o tempo limite expire.
- ❑ shutdownNow(): Tenta parar todas as tarefas em execução ativa, interrompendo as threads, e retorna uma lista das tarefas que estavam aguardando para serem executadas.
- ❑ A partir do Java 19, ExecutorService estende AutoCloseable, tornando o bloco try-with-resources a forma mais limpa e segura de garantir que shutdown() seja chamado. O método shutdownAndAwaitTermination é um padrão robusto recomendado pela documentação oficial do Java para um desligamento mais controlado.

## AULA 03 - COLEÇÕES CONCORRENTES DE ALTA PERFORMANCE

- ❑ Exemplo 3.1: A Falha do Bloqueio de Granularidade Grossa vs. o Poder do ConcurrentHashMap
  - ❑ A forma mais antiga de tornar uma coleção "thread-safe" era usar os wrappers `Collections.synchronized....`. No entanto, essa abordagem cria um gargalo de performance significativo em cenários de alta concorrência.
- ❑ Notas
  - ❑ Este código contrasta diretamente a performance de um `synchronizedMap` com um `ConcurrentHashMap`. O `synchronizedMap` utiliza um único lock (bloqueio) para todo o mapa. Isso significa que apenas uma thread pode acessar o mapa por vez, seja para leitura ou escrita, serializando todas as operações e criando um gargalo de contenção severo.
  - ❑ O `ConcurrentHashMap`, por outro lado, utiliza uma técnica sofisticada chamada fine-grained locking ou lock striping. Em vez de um único lock global, ele divide o mapa em segmentos (ou bins) e utiliza um lock para cada segmento. Isso permite que múltiplas threads escrevam em diferentes partes do mapa simultaneamente. As operações de leitura geralmente são não-bloqueantes e podem ocorrer em paralelo com as escritas. O resultado, como o teste demonstra, é uma escalabilidade e performance muito superiores, tornando o `ConcurrentHashMap` a escolha padrão para mapas concorrentes em Java.

## AULA 03 - COLEÇÕES CONCORRENTES DE ALTA PERFORMANCE

- ❑ Exemplo 3.2: Gerenciando Listeners de Eventos com CopyOnWriteArrayList
  - ❑ Para coleções que são lidas com muito mais frequência do que são modificadas, como listas de listeners de eventos, o CopyOnWriteArrayList oferece uma estratégia de concorrência otimizada.
- ❑ Notas
  - ❑ Este é o caso de uso ideal para CopyOnWriteArrayList. A estratégia "copiar ao escrever" significa que toda operação de modificação (add, set, remove) cria uma cópia inteiramente nova do array subjacente. As operações de leitura, por outro lado, são muito rápidas, pois não exigem locks e operam em um snapshot imutável do array.
  - ❑ Quando a Thread 1 está iterando sobre a lista para notificar os listeners, ela está trabalhando em uma cópia do array que existia no momento em que o laço for-each começou. Quando a Thread 2 chama addListener, uma nova cópia do array é criada, incluindo o novo listener. A Thread 1 não vê essa mudança e completa sua iteração sem erro. As iterações subsequentes verão a lista atualizada. Isso evita a ConcurrentModificationException de forma elegante. Por outro lado, o custo destas modificações é alto, tornando esta coleção inadequada para cenários com escritas frequentes.

## AULA 03 - COLEÇÕES CONCORRENTES DE ALTA PERFORMANCE

- ❑ Exemplo 3.3: O Padrão Produtor-Consumidor com BlockingQueue e Threads Virtuais
  - ❑ O padrão produtor-consumidor é fundamental para desacoplar componentes em sistemas concorrentes. A interface BlockingQueue fornece uma implementação robusta e simples deste padrão.
- ❑ Notas
  - ❑ Este código implementa o padrão produtor-consumidor usando uma BlockingQueue. A beleza desta abordagem é que a coordenação entre as threads é gerenciada inteiramente pela fila, sem a necessidade de wait(), notify() ou synchronized explícitos.
    - ❑ queue.put(item): O produtor chama este método. Se a fila estiver cheia (atingiu sua capacidade de 10), a thread do produtor será bloqueada automaticamente até que o consumidor remova um item e libere espaço.
    - ❑ queue.take(): O consumidor chama este método. Se a fila estiver vazia, a thread do consumidor será bloqueada até que o produtor adicione um novo item.
  - ❑ Essa mecânica de bloqueio fornece um controle de fluxo natural, conhecido como back-pressure. Combinar BlockingQueue com threads virtuais é um padrão extremamente poderoso para construir pipelines de processamento de dados I/O-bound, pois nem o produtor nem o consumidor ocuparão uma thread de plataforma enquanto estiverem bloqueados, esperando pela fila.



## AULA 04 - COORDENAÇÃO AVANÇADA DE THREADS COM SINCRONIZADORES

- ❑ Exemplo 4.1: Coordenando a Inicialização de Serviços com CountdownLatch
  - ❑ Um cenário comum em aplicações de servidor é garantir que todos os serviços essenciais estejam prontos antes de começar a aceitar tráfego. O CountdownLatch é perfeito para isso.
- ❑ Notas
  - ❑ Este exemplo demonstra o uso canônico do CountdownLatch. O latch é inicializado com uma contagem de 3. A thread principal chama latch.await(), o que a bloqueia. Cada serviço, ao concluir sua inicialização, chama latch.countDown(), decrementando o contador. Quando o contador chega a zero, a thread principal é liberada do await() e a aplicação pode prosseguir.
  - ❑ É crucial entender que o CountdownLatch é um mecanismo de uso único (one-shot); uma vez que a contagem chega a zero, ele não pode ser resetado. É ideal para cenários do tipo "espere por N eventos acontecerem uma vez".

## AULA 04 - COORDENAÇÃO AVANÇADA DE THREADS COM SINCRONIZADORES

- ❑ Exemplo 4.2: Sincronizando Trabalho Iterativo com CyclicBarrier
  - ❑ Para algoritmos paralelos que operam em fases, onde um grupo de threads precisa se sincronizar repetidamente antes de avançar, o CyclicBarrier é a ferramenta adequada.
- ❑ Notas
  - ❑ O CyclicBarrier é projetado para cenários onde um grupo fixo de threads precisa se encontrar em um ponto de barreira comum, repetidamente.
  - ❑ No exemplo, 3 workers executam a Fase 1 e chamam `barrier.await()`. As duas primeiras threads a chegar ficarão bloqueadas. Quando a terceira thread chega, a barreira é "quebrada", a `barrierAction` opcional é executada (pela última thread a chegar), e todas as três threads são liberadas para iniciar a Fase 2. A barreira então se reseta automaticamente, pronta para a próxima fase. É "cíclica" porque pode ser reutilizada, ao contrário do `CountDownLatch`.

## AULA 04 - COORDENAÇÃO AVANÇADA DE THREADS COM SINCRONIZADORES

- ❑ Exemplo 4.3: Limitando o Acesso a Recursos com Semaphore
  - ❑ Um semáforo é usado para controlar o acesso a um recurso compartilhado, limitando o número de threads que podem acessá-lo simultaneamente.
- ❑ Notas
  - ❑ Este código implementa um pool de recursos limitado por um Semaphore. O semáforo é inicializado com 3 "permissões". Uma thread que deseja usar o recurso deve primeiro chamar `semaphore.acquire()`. Se uma permissão estiver disponível, a thread a obtém e continua. Se todas as 3 permissões estiverem em uso, a thread ficará bloqueada até que outra thread chame `semaphore.release()`. É absolutamente crítico que `release()` seja chamado em um bloco `finally` para garantir que a permissão seja devolvida mesmo que ocorra uma exceção, evitando deadlocks.
  - ❑ Este padrão é especialmente relevante na era das threads virtuais. Embora possamos criar milhões de threads virtuais, os recursos que elas acessam (como conexões de banco de dados) geralmente são limitados. O semáforo é a ferramenta moderna e correta para limitar a concorrência a esses recursos, conforme mencionado na aula sobre threads virtuais.

## AULA 04 - COORDENAÇÃO AVANÇADA DE THREADS COM SINCRONIZADORES

- ❑ Exemplo 4.4: Coordenação Multifásica e Dinâmica com Phaser
  - ❑ O Phaser é o sincronizador mais flexível, combinando e expandindo as funcionalidades do CountdownLatch e do CyclicBarrier, permitindo que o número de participantes mude dinamicamente entre as fases.
- ❑ Notas
  - ❑ O Phaser se destaca por sua capacidade de lidar com um número dinâmico de participantes (parties).
  - ❑ No exemplo, todas as 4 tarefas se registram no início. Após a primeira fase, as tarefas ímpares se desregistram usando `arriveAndDeregister`, enquanto as tarefas pares continuam para a próxima fase.
  - ❑ Isso seria impossível de implementar de forma limpa com um CyclicBarrier, que exige um número fixo de participantes. O Phaser é ideal para computações paralelas mais complexas e hierárquicas, onde o conjunto de tarefas participantes pode evoluir ao longo do tempo.

## AULA 05 - DECOMPOSIÇÃO PARALELA COM O FRAMEWORK FORK/JOIN

- ❑ Exemplo 5.1: Processamento de Dados com RecursiveTask
  - ❑ O exemplo canônico para o Fork/Join é a soma de elementos em um grande array. A tarefa é dividida recursivamente até que os pedaços sejam pequenos o suficiente para serem somados sequencialmente.
- ❑ Notas
  - ❑ Este código implementa a lógica de "dividir para conquistar" descrita na aula. A classe `SumTask` estende `RecursiveTask<Long>`, indicando que ela retorna um resultado do tipo `Long`. O método `compute()` contém a lógica principal:
    - ❑ Caso Base: Se o segmento do array é menor que um `THRESHOLD`, ele é somado sequencialmente.
    - ❑ Caso Recursivo: Se for maior, a tarefa é dividida em duas.
  - ❑ A otimização `leftTask.fork(); rightTask.compute(); leftTask.join();` é crucial. Em vez de fazer `fork()` em ambas as tarefas e depois `join()` em ambas, a thread de trabalho atual executa `rightTask.compute()` diretamente. Isso mantém a thread ocupada. Enquanto isso, `leftTask` é colocada na fila de trabalho (deque) da thread atual, onde pode ser "roubada" por outra thread de trabalho ociosa. Este algoritmo de `work-stealing` é o que torna o `ForkJoinPool` eficiente, garantindo que todos os núcleos da CPU permaneçam ocupados.

## AULA 05 - DECOMPOSIÇÃO PARALELA COM O FRAMEWORK FORK/JOIN

- ❑ Exemplo 5.2: Usando o Pool Comum e Pools Customizados
  - ❑ O ForkJoinPool oferece um pool estático compartilhado, o `commonPool`, para conveniência. No entanto, para aplicações complexas, pode ser necessário criar pools customizados.
- ❑ Notas
  - ❑ Este exemplo mostra as duas formas de usar o ForkJoinPool.
    - ❑ `ForkJoinPool.commonPool()`: É um pool estático e compartilhado, usado por padrão por Parallel Streams e algumas operações de `CompletableFuture`. Sua conveniência é também seu maior risco: uma tarefa mal comportada (por exemplo, uma que bloqueia em I/O) pode monopolizar uma thread do pool comum, afetando negativamente outras partes da aplicação que dependem dele.
    - ❑ `new ForkJoinPool(parallelism)`: Criar uma instância customizada fornece isolamento. É a abordagem recomendada para tarefas Fork/Join de longa duração ou críticas, garantindo que elas não interfiram com o resto do sistema.
  - ❑ O framework Fork/Join não é apenas mais um utilitário de concorrência; ele é a fundação arquitetural para APIs de paralelismo de mais alto nível em Java. Compreender seu mecanismo de work-stealing é a chave para entender as características de performance (e as armadilhas potenciais) dos Parallel Streams. A aula sobre Fork/Join fornece o modelo mental necessário para diagnosticar problemas com Parallel Streams posteriormente, pois as propriedades do motor (Fork/Join) "vazam" através da abstração (Streams).

## AULA 06 - PROGRAMAÇÃO ASSÍNCRONA COM COMPLETABLEFUTURE

- ❑ Exemplo 6.1: A Analogia "Map vs. FlatMap": thenApply vs. thenCompose
  - ❑ A distinção entre thenApply e thenCompose é um dos conceitos mais importantes e frequentemente confusos do CompletableFuture. A analogia com os métodos map e flatMap da API de Streams é a melhor forma de entendê-la.
- ❑ Notas
  - ❑ Este exemplo demonstra a diferença fundamental entre thenApply e thenCompose.
    - ❑ thenApply(T -> U): É usado para uma transformação síncrona do resultado de um CompletableFuture. É análogo ao Stream.map(). Se a função de transformação retorna um CompletableFuture, o resultado será um CompletableFuture aninhado (CompletableFuture<CompletableFuture<Profile>>), o que é inconveniente.
    - ❑ thenCompose(T -> CompletableFuture<U>): É usado para encadear uma operação assíncrona que depende do resultado da anterior. É análogo ao Stream.flatMap(). Ele "achata" o resultado, evitando o aninhamento e retornando um CompletableFuture<Profile> simples, facilitando o encadeamento de mais operações.
  - ❑ A regra geral é: se a sua função de encadeamento retorna um valor diretamente, use thenApply. Se ela retorna outro CompletableFuture, use thenCompose.

## AULA 06 - PROGRAMAÇÃO ASSÍNCRONA COM COMPLETABLEFUTURE

- ❑ Exemplo 6.2: Combinando Resultados Assíncronos Independentes com thenCombine
  - ❑ Quando duas tarefas assíncronas podem ser executadas em paralelo e seus resultados precisam ser combinados, thenCombine é a escolha certa.
- ❑ Notas
  - ❑ Este código demonstra o thenCombine. As chamadas getOrderHistory() e getShippingPreferences() são disparadas e executam em paralelo. thenCombine espera que ambas as tarefas sejam concluídas. Quando isso acontece, ele executa a BiFunction fornecida, que recebe os resultados de ambas as tarefas como argumentos e produz um único resultado combinado.
  - ❑ É importante diferenciar de thenCompose, que é para operações assíncronas sequenciais e dependentes. thenCombine é para operações paralelas e independentes.



## AULA 06 - PROGRAMAÇÃO ASSÍNCRONA COM COMPLETABLEFUTURE

- ❑ Exemplo 6.3: Orquestrando e Agregando Múltiplas Tarefas com allOf
  - ❑ Para cenários onde é preciso esperar pela conclusão de um número variável de tarefas assíncronas, `CompletableFuture.allOf` é a ferramenta de sincronização.
- ❑ Notas
  - ❑ Este exemplo mostra como usar `CompletableFuture.allOf` para sincronização.
  - ❑ Um ponto crucial é que `allOf` retorna um `CompletableFuture<Void>`. Seu único propósito é ser concluído quando todos os `CompletableFuture`s fornecidos terminarem. Ele não agrega os resultados.
  - ❑ O padrão correto para coletar os resultados é encadear um `thenApply` no `allDoneFuture`. Dentro do `thenApply`, iteramos sobre a lista original de futures e chamamos `join()` em cada um.
  - ❑ A chamada `join()` não bloqueará de forma significativa, pois o `thenApply` só é executado depois que todas as tarefas já foram concluídas.

## AULA 06 - PROGRAMAÇÃO ASSÍNCRONA COM COMPLETABLEFUTURE

- ❑ Exemplo 6.4: Implementando Pipelines Resilientes com `exceptionally` e `handle`
  - ❑ Aplicações assíncronas robustas precisam de um tratamento de erros eficaz. `CompletableFuture` fornece mecanismos para isso, análogos aos blocos `try-catch-finally`.
- ❑ Notas
  - ❑ Este código demonstra as duas principais formas de tratamento de erros no `CompletableFuture`.
    - ❑ `exceptionally(Function<Throwable, T>)`: É o equivalente assíncrono de um bloco `catch`. Ele é acionado apenas se o estágio anterior for concluído com uma exceção. A função recebe a exceção e pode retornar um valor de fallback do mesmo tipo do `CompletableFuture`, permitindo que o pipeline se recupere e continue.
    - ❑ `handle(BiFunction<T, Throwable, U>)`: É mais geral, análogo a um bloco `finally` que pode transformar o resultado. Ele é sempre executado, independentemente de o estágio anterior ter tido sucesso ou falha. A `BiFunction` recebe dois argumentos: o resultado (que será `null` se houver erro) e a exceção (que será `null` se houver sucesso). Isso permite processar ambos os cenários em um único lugar.
  - ❑ O `CompletableFuture` introduziu padrões de composição funcional na concorrência Java, permitindo que os desenvolvedores pensem em fluxos de trabalho concorrentes como pipelines de dados. No entanto, essa mesma expressividade pode levar a cadeias de chamadas complexas e difíceis de depurar.
  - ❑ Este é exatamente o problema que as `Threads Virtuais` e a `Concorrência Estruturada` visam resolver, trazendo a concorrência de volta a uma estrutura léxica mais simples, com aparência sequencial. Portanto, o `CompletableFuture` pode ser visto tanto como uma ferramenta poderosa quanto como um artefato histórico das restrições da era pré-Loom.

## AULA 07 - O PODER E AS ARMADILHAS DOS PARALLEL STREAMS

- ❑ Exemplo 7.1: O Caso de Uso Ideal: Operações CPU-Intensive em Grandes Conjuntos de Dados Divisíveis
  - ❑ Quando usados corretamente, os Parallel Streams podem proporcionar ganhos de performance significativos.
- ❑ Notas
  - ❑ Este código demonstra um cenário onde `parallelStream()` brilha. As condições ideais foram atendidas :
    - ❑ Tarefa CPU-Bound: A verificação de primalidade (`isPrime`) é computacionalmente intensiva.
    - ❑ Grande Volume de Dados: O overhead do paralelismo (dividir a tarefa, coordenar as threads) só é compensado em conjuntos de dados grandes. O modelo NQ (N = número de elementos, Q = custo por elemento) ajuda a avaliar isso.
    - ❑ Fonte Divisível: `ArrayList` é uma fonte ideal porque pode ser dividida em sub-listas de forma eficiente e barata. Fontes como `LinkedList` são péssimas para paralelismo.
    - ❑ Operações Stateless: A função `isPrime` é stateless (sem estado); seu resultado depende apenas de sua entrada, e ela não modifica nenhum estado compartilhado.
  - ❑ Neste caso, o ganho de performance é real e mensurável.

## AULA 07 - O PODER E AS ARMADILHAS DOS PARALLEL STREAMS

- ❑ Exemplo 7.2: Antipattern - O Perigo do I/O Bloqueante em Parallel Streams
  - ❑ Tentar usar `parallelStream()` para acelerar operações de I/O é um dos erros mais comuns e perigosos.
- ❑ Notas
  - ❑ Este exemplo ilustra um antipadrão crítico. Os Parallel Streams utilizam o `ForkJoinPool.commonPool()` compartilhado, que por padrão tem um número de threads de plataforma igual ao número de núcleos da CPU (ou `num_cores - 1`).
  - ❑ Se uma tarefa dentro do stream realiza uma operação de I/O bloqueante (simulada aqui com `Thread.sleep()`), ela monopoliza uma dessas preciosas threads de plataforma, que fica ociosa esperando.
  - ❑ Se todas as threads do `commonPool` forem bloqueadas por operações de I/O, o pool inteiro fica saturado. Isso não apenas impede que o `parallelStream` escale, mas também pode paralisar outras partes da aplicação que dependem do `commonPool`, como `CompletableFutures` ou outras tarefas `Fork/Join`.
  - ❑ Para paralelismo de I/O, threads virtuais ou `CompletableFuture` com um executor customizado são as soluções corretas.

## AULA 07 - O PODER E AS ARMADILHAS DOS PARALLEL STREAMS

- ❑ Exemplo 7.3: Antipattern - Condições de Corrida com Lambdas Stateful
  - ❑ As funções lambda usadas em operações de stream devem ser stateless. Modificar um estado compartilhado de dentro de um `parallelStream` é uma receita para resultados incorretos.
- ❑ Notas
  - ❑ Este código demonstra o perigo de lambdas com estado (stateful lambdas). A primeira tentativa de adicionar elementos a um `ArrayList` padrão de dentro de um `forEach` paralelo resultará em uma condição de corrida. Múltiplas threads tentarão modificar a estrutura interna do `ArrayList` ao mesmo tempo, levando a resultados inconsistentes (elementos perdidos) e potencialmente a exceções.
  - ❑ As operações de stream devem ser livres de efeitos colaterais. A maneira correta de agregar resultados de um stream paralelo é usar as operações de terminal projetadas para isso, como os `Collectors`. O `Collectors.toList()` usa internamente um mecanismo thread-safe para combinar os resultados parciais de cada thread em uma lista final.
  - ❑ A decisão de projeto de fazer o `parallelStream()` usar um `commonPool` global e compartilhado é a causa raiz de suas maiores armadilhas. Embora conveniente, cria um cenário de "tragédia dos comuns", onde um stream mal comportado pode impactar negativamente toda a aplicação. Isso torna o `parallelStream()` uma ferramenta altamente especializada, não um acelerador de propósito geral.

## AULA 08 - A REVOLUÇÃO DAS THREADS VIRTUAIS

- ❑ Exemplo 8.1: Construindo uma Aplicação Massivamente Concorrente com Threads Virtuais
  - ❑ As threads virtuais tornam o modelo "uma thread por requisição" não apenas viável, mas a abordagem recomendada e escalável para aplicações I/O-bound.
- ❑ Notas
  - ❑ Este exemplo cumpre a principal promessa das threads virtuais: escalabilidade massiva com código simples e bloqueante. O código parece sequencial e é fácil de ler e depurar, mas por baixo dos panos, a JVM está gerenciando eficientemente 200.000 tarefas concorrentes.
  - ❑ Antes do Projeto Loom, alcançar essa escala exigiria programação assíncrona complexa com callbacks ou CompletableFutures. Agora, o modelo "uma thread por tarefa" é a forma mais simples e performática de lidar com cargas de trabalho I/O-bound. A thread deixa de ser um recurso físico escasso para se tornar um objeto lógico barato.

**ATÉ A PRÓXIMA!**